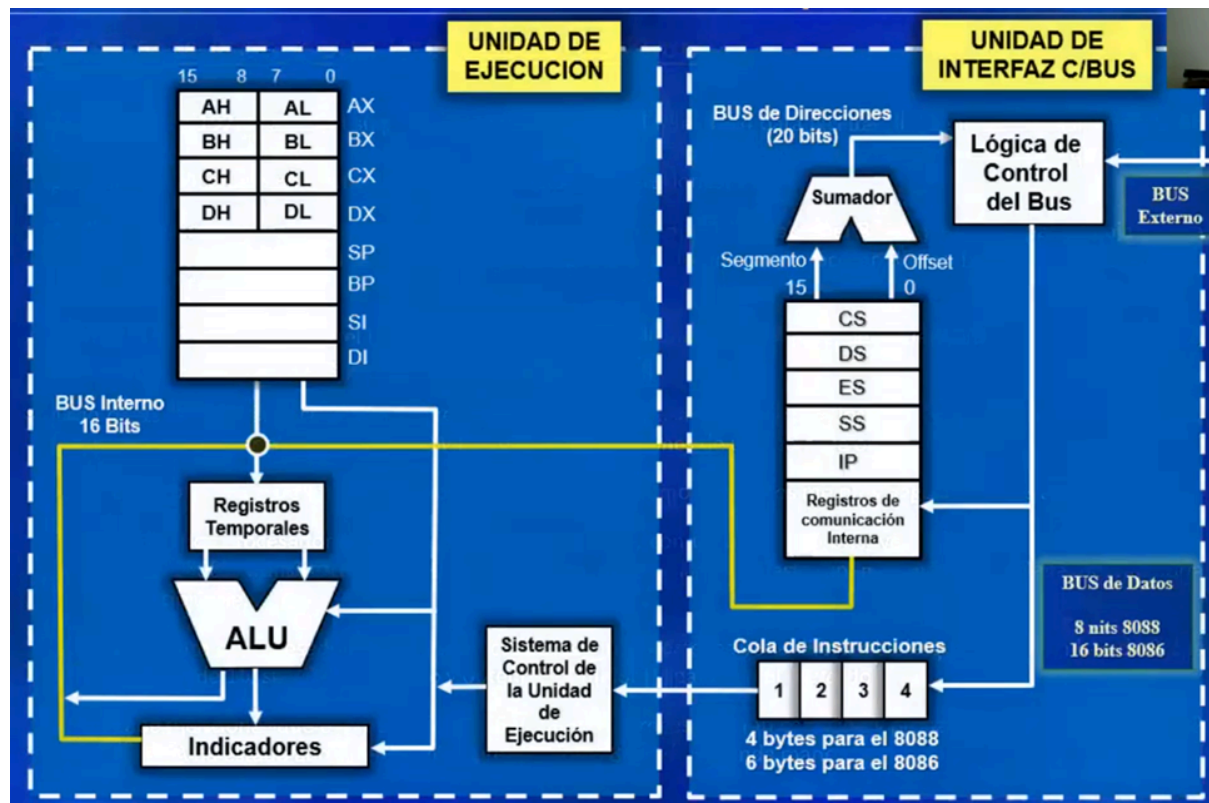


8. Arquitectura 8086



▼ Unidad de Ejecución

Se encarga de ejecutar las instrucciones obtenidas de la **Cola de Instrucciones**, realizar los cálculos mediante la **ALU**, gestionar los **Registros de Propósito General**, operar con la pila y controlar el flujo de ejecución del programa.

▼ Registros de Propósito General

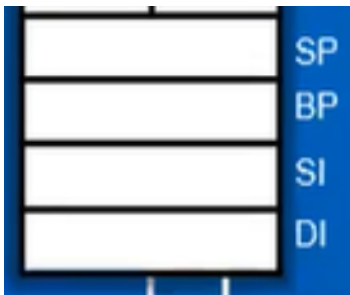
Son 4 registros de 16 bits vinculados estrechamente a la **ALU**. Además, los **registros de propósito general** pueden dividirse en 2 registros de 8 bits:

- **AH (High):** almacena los 8 **bits de mayor peso** (bits 8 a 15).
- **AL (Low):** almacena los 8 **bits de menor peso** (bits 0 a 7).



- **AX (Acumulador):** registro principal para operaciones aritméticas y lógicas. Muy utilizado en instrucciones de **Entrada/Salida** y en **multiplicación y división**.
- **BX (Base):** puede emplearse como **registro base** para **direccionamiento indexado** y para cálculos. Suele almacenar el **OFFSET** de estructuras de datos.
- **CX (Contador):** se utiliza en cálculos, pero principalmente para **controlar repeticiones y bucles** (instrucción LOOP).
- **DX (Datos):** requerido en algunas operaciones de **Entrada/Salida** y en multiplicación/división de mayor tamaño, trabajando junto con AX.

▼ Registros de Direcciones de Datos (Apuntadores)



- **SP (Puntero de Pila):** Aunque es un registro de uso general, debe utilizarse sólo como **puntero de pila**, el cual sirve para almacenar las direcciones de retorno de subrutinas y los datos temporarios (mediante las instrucciones PUSH y POP).
 - Al introducir (PUSH) un valor en la **pila** a este registro se le resta 2.
 - Al extraer (POP) un valor de la **pila** este a registro se le suma 2.
- **BP (Puntero de Base):** Generalmente se utiliza para realizar **direccionamiento indirecto** dentro de la **pila**.
- **SI (Índice Fuente):** Sirve como **puntero fuente** para las **operaciones con cadenas**. También sirve para realizar **direccionamiento indirecto**. Lleva la **dirección donde comienzan** los datos que se quieran leer dentro de un **segmento de datos**.

- **DI (Índice Destino):** Sirve como **puntero destino** para las **operaciones con cadenas**. También sirve para realizar **direccionamiento indirecto**. Lleva la **dirección donde terminan** los datos que se quieran leer dentro de **un segmento de datos**.

▼ Registros Temporales



Almacenan valores intermedios durante cálculos en la ALU. Actúan como un "**buffer**" interno permitiendo **aislar y sincronizar** los datos provenientes de los **Registros de Propósito General** antes de ser procesados por la ALU.

▼ ALU (Unidad Aritmético-Lógica)



Realiza operaciones **aritméticas y lógicas**, y actualiza los **Registros Banderas (FLAGS)** de acuerdo con el resultado de las operaciones.

▼ Registros Banderas (Flags)



Están vinculados a la ALU y a los **Registros de Propósito General**, y almacenan información de estado y de control del procesador. Permiten que las **instrucciones de bifurcación** (saltos condicionales) tomen

decisiones basándose en los resultados temporales almacenados en estos registros.

El **Registro Banderas (Flags)** es de 16 bits, de los cuales solo 9 se utilizan.

				OF	DF	IF	TF	SF	ZF		AF		PF		CF
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00

▼ **Banderas de Estado:**

6 **banderas de estado** registran el estado del procesador luego de una operación aritmética o comparación.

1. **CF (Carry Flag)**: Indica acarreo en operaciones aritméticas.
2. **OF (Overflow Flag)**: Indica desbordamiento aritmético.
3. **ZF (Zero Flag)**: Se activa si el resultado es 0 o una comparación es igual.
4. **SF (Sign Flag)**: Indica si el resultado es negativo o una comparación es no igual.
5. **PF (Parity Flag)**: Verifica la transferencia de bytes entre 2 componentes de la computadora.
6. **AF (Auxiliar Flag)**: Indica si es necesario un ajuste en operaciones aritméticas con números BCD.

▼ **Banderas de Control:**

3 **banderas de control** que registran el modo de funcionamiento de la computadora.

1. **DF (Direction Flag)**: Controla la dirección (hacia adelante o hacia atrás) en las **operaciones con cadenas de caracteres** incrementando o decrementando automáticamente los **registros índices** (SI y DI).
2. **IF (Interrupt Flag)**: Indica si están disponibles o no las interrupciones de los dispositivos externos.
3. **TF (Trap Flag)**: Controla la operación modo paso a paso (usada por el programa DEBUG).

▼ **Sistema de Control de la Unidad de Ejecución**



Se encarga de ir sacando las **Instrucciones** de la **Cola de Instrucciones**, decodificarlas y ejecutarlas. Genera las señales de control que sincronizan la **ALU**, los **Registros de Propósito General** y otros componentes internos.

▼ Unidad de Interfaz C/BUS

Es la encargada de comunicar al **Procesador** con la **Memoria Central** y los dispositivos de E/S. Su función principal es realizar todo el ciclo **Fetch** (búsqueda) de instrucciones y operandos para mantener alimentada a la Unidad de Ejecución.

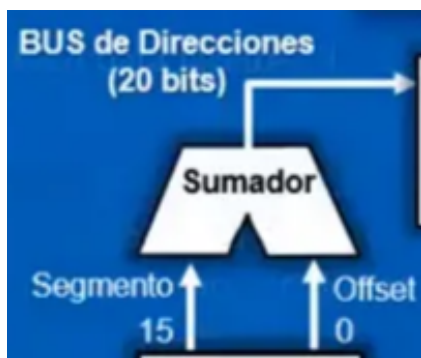
▼ Lógica de Control del Bus



Se encarga de recibir las instrucciones desde la memoria y enviarlas a la **Cola de Instrucciones**. Gestiona la conexión física con el **Bus Externo**.

▼ Sumador

Este componente es vital porque el 8086 tiene registros de 16 bits y un **Bus de Direcciones** de 20 bits. El **Sumador** obtiene la **Dirección Efectiva del Operando (20 bits)** mediante el proceso de **Suma Desplazada**.



Cálculo:

1. Toma la dirección base del **Registro de Segmento** y la multiplica por 10H (desplaza 4 bits a la izquierda, generando un 0 a la derecha).
2. Le suma el valor del **Desplazamiento (OFFSET)**.

Fórmula: $(Segmento \times 10H) + OFFSET = \text{Dirección Física.}$

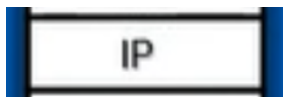
▼ Registros de Direcciones de Segmentos

Son registros de 16 bits dedicados a direccionar las ubicaciones de Memoria.

Cada segmento puede direccionar un máximo de 64 **KBytes** ($2^{16} = 65536$).



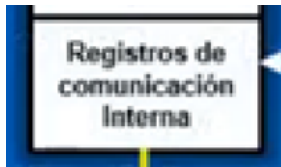
- **CS (Segmento de Código):** se utiliza para direccionar la ubicación de la Memoria en el **Segmento de Código de la Memoria**, donde se almacena el **programa ejecutable**.
- **DS (Segmento de Datos):** apunta al **Segmento de Datos de la Memoria** donde se almacenan los **datos**.
- **ES (Segmento Extra):** también se refiere a un **Segmento en la Memoria de la Zona Alta** disponible para almacenar información.
- **SS (Segmento de Pila):** se utiliza para direccionar el **Segmento de Pila de la Memoria**. El **Segmento de Pila** es el segmento de memoria que se utiliza para almacenar **datos de pila**.



- **IP (Puntero de Instrucción):** especifica el **desplazamiento (OFFSET)** de la siguiente instrucción a ejecutar, relativo al segmento CS.
 - Dirección lógica: [CS : IP].
 - Raramente se referirá directamente a IP. En lugar de eso, se usarán instrucciones para

cambiar IP alterando la localización de la **siguiente Instrucción** a ejecutarse y cambiando el flujo del **Programa**.

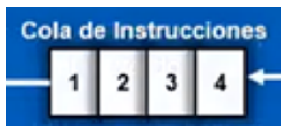
▼ Registros de Comunicación Interna



Manejan la información dentro del **Procesador** y actúan como intermediarios (**buffers**) entre la **Unidad de Ejecución** y la **Unidad de Interfaz C/Bus**.

▼ Cola de Instrucciones

Es una estructura F.I.F.O. que almacena las instrucciones traídas de Memoria ("**prefetch**") antes de ejecutarlas.

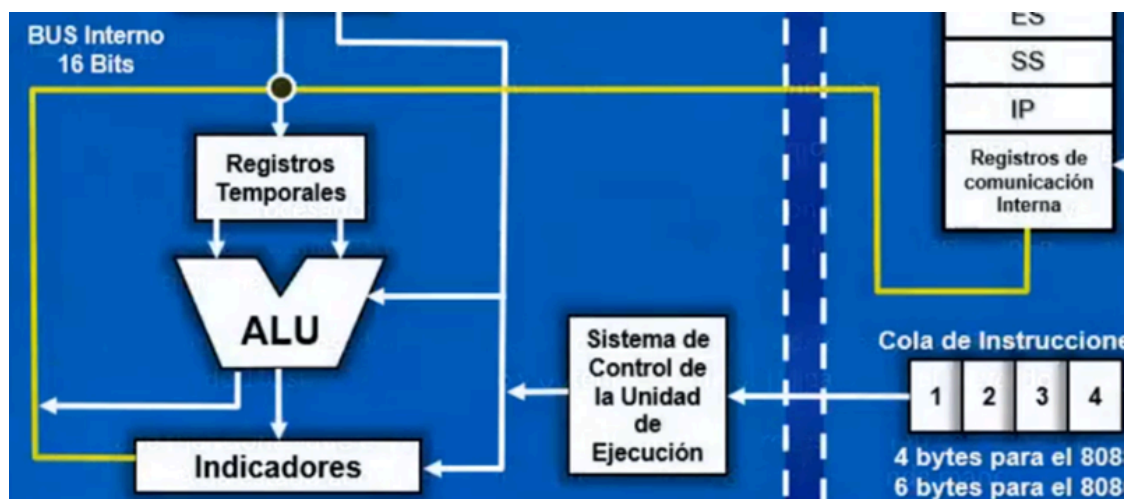


- **Capacidad (Diferencia Clave):**

- 8086: Almacena **6 Bytes**.
- 8088: Almacena **4 Bytes**.

Permite solapar la búsqueda de la siguiente instrucción mientras se ejecuta la actual (principio de **Pipeline**).

▼ BUS Interno



Interconecta la **Unidad de Ejecución** y la **Unidad de Interfaz C/BUS**, permitiendo el flujo de **Instrucciones** y **Datos** dentro del **Procesador**.

▼ Ciclo FETCH (Búsqueda)

1. La instrucción ingresa a la **Lógica de Control del Bus o Controladora del Bus**, encargado de comunicar al procesador con el exterior., en este caso con la **Memoria RAM**. Recibe todas las instrucciones del **Bus Externo** y las envía a la **Cola de Instrucciones**.
2. La **Cola de Instrucciones** es una estructura FIFO de 6 palabras encargada de ir almacenando en orden de llegada cada instrucción de la **Memoria Central** (conocido como **ciclo prefetch**).
3. **Sistema de control de la Unidad de Ejecución** se encarga de ir sacando las instrucciones de la cola, decodificarlas y ejecutarlas.
4. Las operaciones de la ALU generan una serie de eventos, registrados en el **Registro Indicadores de Eventos o bits Bandera**, que almacenan:
 - 1 si se produce el evento.
 - 0 si no se produce el evento.
5. Para realizar las Operaciones Aritméticas, la ALU utiliza a los **Registros de Propósito General**, los que almacenan los valores temporales que la instrucción necesita.
6. Si la instrucción requiere acceso a operando en memoria, pasan al entorno del **Sumador**, el que obtiene la **Dirección Efectiva del operando** sumando el valor del registro DS (**Segmento de Datos**) mas el valor de **desplazamiento**.
 - El cálculo para obtener la dirección real en memoria se realiza en 2 pasos:
 1. **Desplazamiento del Segmento**: El valor del **Registro de Segmento** (ej. DS) se multiplica por 10H (se agregan 4 ceros binarios o un 0 hexadecimal a la derecha).
 2. **Suma del OFFSET**: A ese valor desplazado se le suma el **Desplazamiento (OFFSET)**.

Fórmula: $(\text{Segmento} \times 10H) + \text{Offset} = \text{Dirección Física}$

▼ Modos de Direccionamiento

Los **Modos de Direccionamiento** indican la manera de obtener los operandos de **Memoria**. El tipo de direccionamiento se determina en función de los operandos de la instrucción, siendo el mas usado **MOV destino, origen**.

La instrucción MOV realiza transferencia de datos desde un **operando origen** a un **operando destino**.

Su formato es el siguiente: MOV destino, origen

1 - Direccionamiento de Registro

Se lo utiliza cuando ambos operando son registros de la arquitectura.

```
MOV AX,BX ; transfiere el contenido de BX en AX
```

2 - Direccionamiento Inmediato

Cuando el operando origen es una constante.

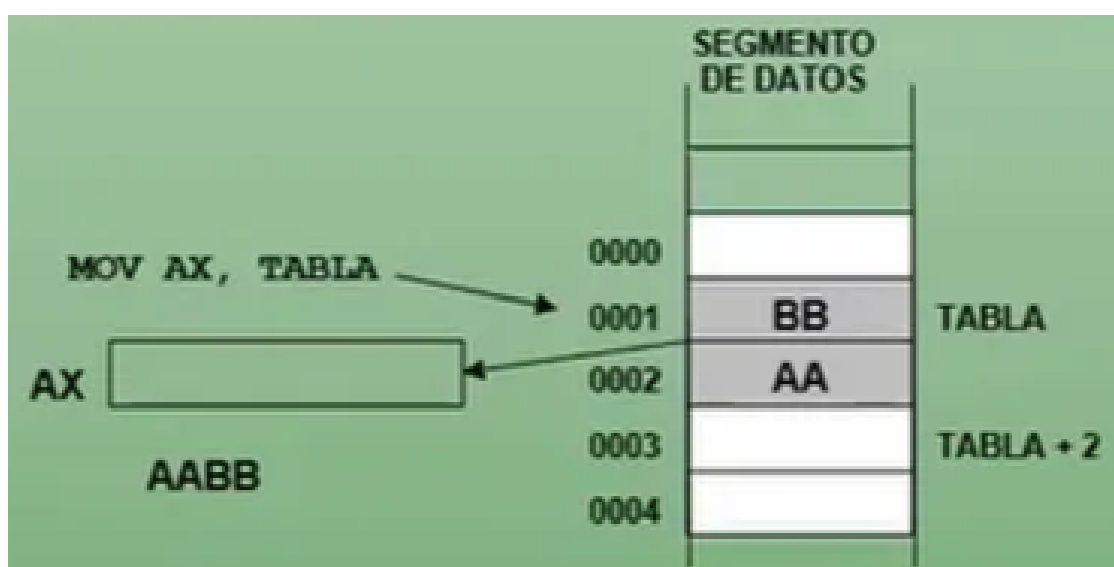
```
MOV AX,500 ;carga en AX el valor 500
```

3 - Direccionamiento Directo

Cuando el operando es una dirección de Memoria. Ésta puede ser especificada con su valor entre [], o bien mediante una variable definida previamente (cómo definir etiquetas se verá más adelante).

```
MOV BX, [1000] ; almacena en BX el contenido de la dirección de Memoria DS:1000.
```

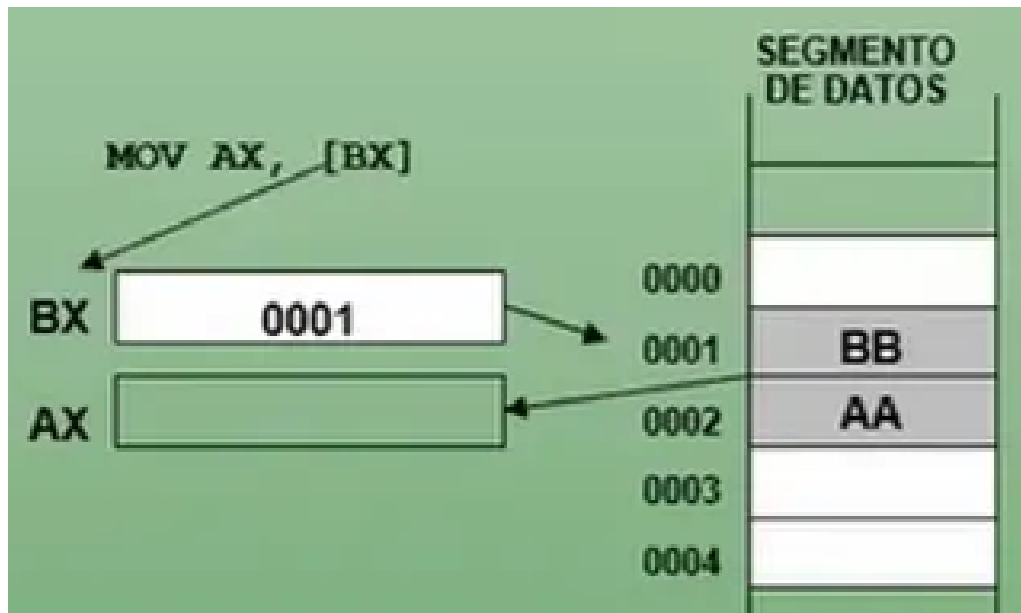
```
MOV AX, TABLA ; almacena en AX el contenido de la dirección de Memoria DS:TABLA.
```



4 - Direccionamiento Indirecto mediante Registro

Cuando el operando esta en memoria en una posición contenida en un registro (BX, BP, SI o DI).

`MOV AX,[BX]` ; almacena en AX el contenido de la **dirección de Memoria** DS:[BX].

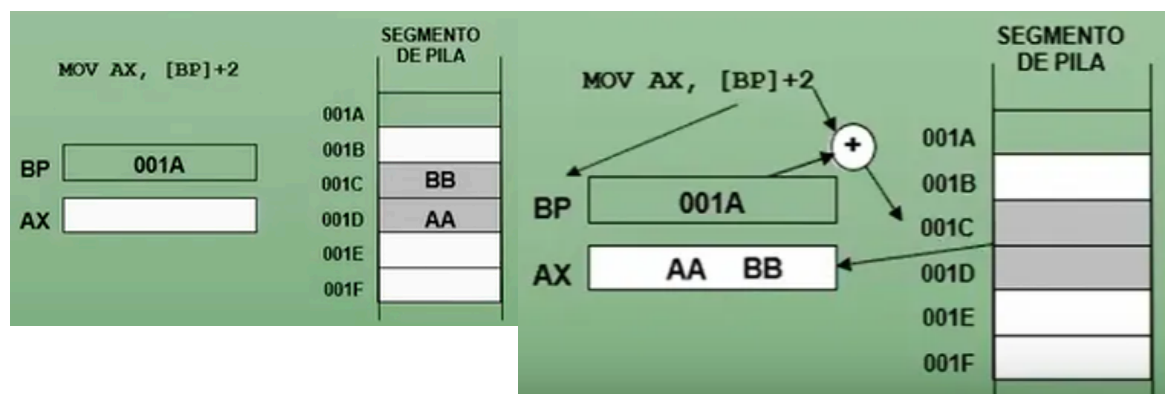


5 - Direccionamiento Indirecto por Registro Base

Cuando el operando esta en memoria en una posición apuntada por el registro BX o BP al que se le añade un determinado **desplazamiento**. Permite acceder ,de una forma cómoda, a **estructuras de datos** que se encuentran en memoria.

`MOV AX, [BP] + 2` ; almacena en AX el contenido de la posición de memoria que resulte de sumar 2 al contenido de BP (dentro de **Segmento de Pila**).

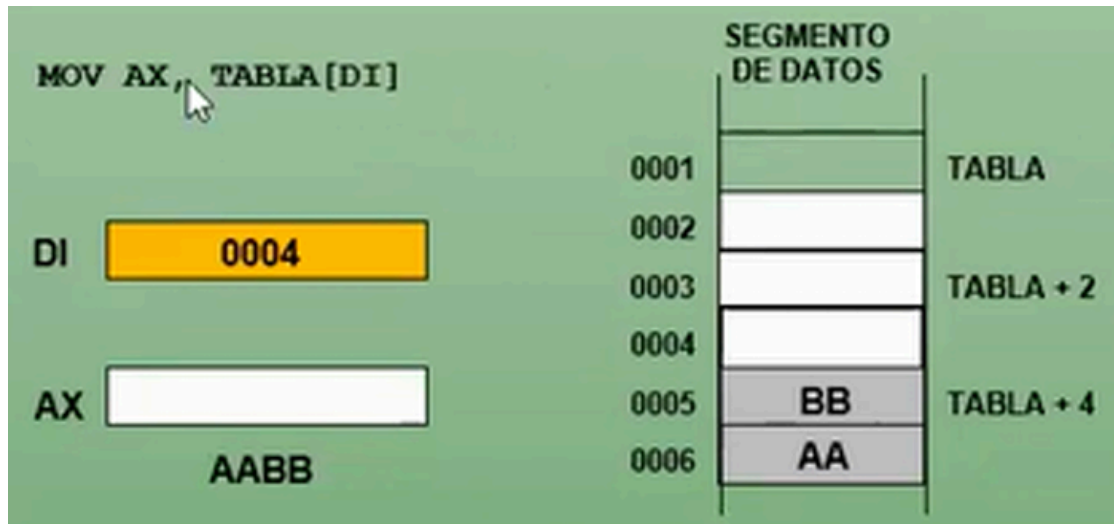
Equivalente a `MOV AX, [BP + 2]`



6 - Direccionamiento Indexado

Cuando la dirección del operando es obtenida como la suma de un **desplazamiento** más un **índice** (DI, SI).

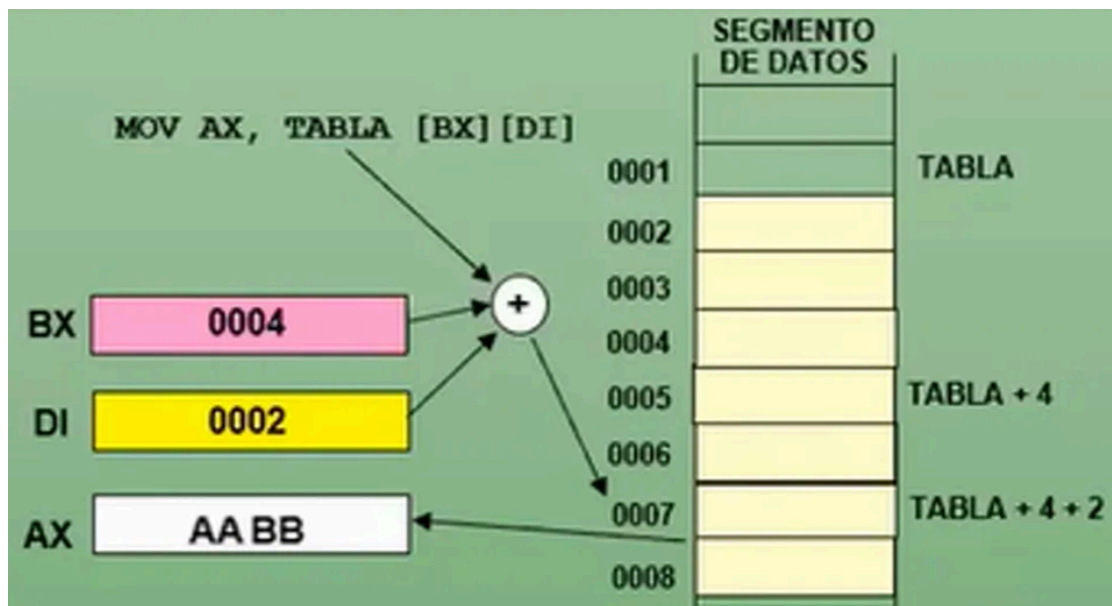
MOV AX, TABLA[DI] ; almacena en AX el contenido de la posición de memoria apuntada por el resultado de sumarle a TABLA el contenido de DI



7 - Direccionamiento Indexado respecto a una Base

Cuando la dirección del operando se obtiene de la suma de un **Registro Base** (BP o BX), de un **índice** (DI, SI) y opcionalmente un **desplazamiento**.

MOV AX, TABLA[BX][DI] ; almacena en AX el contenido de la posición de memoria apuntada por la suma de TABLA, el contenido de BX y el contenido de DI



Resumen de los Modos de Direccionamiento

▼ Se suele pedir un Ejemplo

Ejemplificación del funcionamiento con instrucción:

PUSH

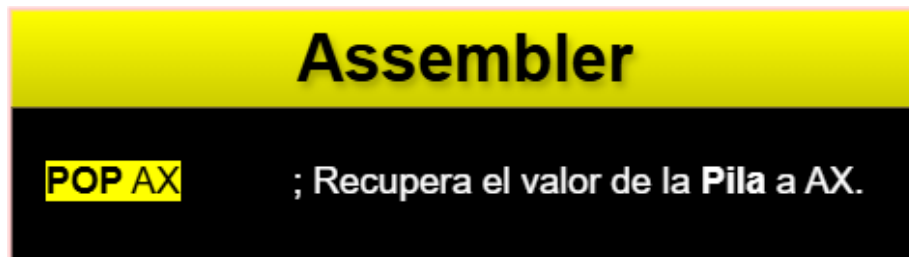
- **Función:** Almacena un valor en la **Pila**.
- **Flujo de operación:**
 1. **Disminuye el valor** del **Puntero de Pila (SP)** en 2.
 2. Se copia el contenido del **Registro** o **dato** especificado a la **dirección de Memoria** apuntada por el nuevo valor de **SS**.

Assembler	
<code>MOV AX, 123h</code>	; Carga el valor 123h en el registro AX.
<code>PUSH AX</code>	; Guarda el valor de AX en la Pila .

- Antes del **PUSH**:
 - $SP = 0xFFFFE$ (supongamos **Pila** inicial en **segmento SS**).
 - **Memoria**: vacío.
- Después del **PUSH**:
 - $SP = 0xFFFFC$.
 - **Memoria** en **SS**:
 - $0xFFFFC = 123h$.

POP

- **Función**: Extrae un valor de la **Pila** al registro o ubicación especificada.
- **Flujo de operación**:
 1. Copia el contenido de la dirección apuntada por **SP** al registro/destino especificado.
 2. Incrementa el valor de **SP** en 2.



- Antes del **POP**:
 - $SP = 0xFFFFC$.
 - **Memoria** en **SS**:
 - $0xFFFFC = 123h$.
- Después del **POP**:
 - $AX = 123h$.
 - $SP = 0xFFFFE$.

PUSH y POP

- **PUSH**: Almacena temporalmente el valor de un registro (en este caso, AX) en la **Pila**.

- **POP**: Recupera el valor almacenado en la **Pila** y lo coloca nuevamente en un registro (en este caso, también AX).
- **Uso combinado**: Sirve para guardar valores temporalmente y restaurarlos después, lo que es útil en operaciones anidadas o para preservar datos durante llamadas a subrutinas.

Assembler	
MOV AX, 123h	; Cargar 123h en AX.
PUSH AX	; Empujar 123h a la Pila .
MOV AX, 0000h	; Limpiar AX.
POP AX	; Recuperar 123h desde la Pila a AX.

Flujo de operación

1. Estado inicial:
 - $SP = FFEH$ (puntero de **Pila** inicial).
 - $AX = 0000h$ (estado inicial del registro AX).
2. MOV AX, 123h:
 - Carga 123h en AX.
 - Registro AX:
 - $AX = 123h$.
3. **PUSH** AX:
 - Decrementa **SP** en 2:
 - $SP = FFECh$.
 - Almacena AX en la dirección apuntada por **SP**:
 - $[SS : FFECh] = 123h$.
 - Estado de la **Pila**:
 - $[SS : FFECh] = 123h$.

4. MOV AX, 0000h:

- Limpia el registro AX.
- Registro AX:
 - $AX = 0000h$.

5. POP AX:

- Extrae el valor de la pila (123h) a AX.
- Incrementa SP en 2:
 - $SP = FFEeh$.
- Registro AX:
 - $AX = 123h$.
- Estado de la Pila: La Pila vuelve a estar vacía.